

Analyzing the Evolution and Maintenance of ML Models on Hugging Face

Joel Castaño

Universitat Politècnica de Catalunya
joel.castano@upc.edu

Xavier Franch

Universitat Politècnica de Catalunya
xavier.franch@upc.edu

Silverio Martínez-Fernández

Universitat Politècnica de Catalunya
silverio.martinez@upc.edu

Justus Bogner

Vrije Universiteit Amsterdam
j.bogner@vu.nl

ABSTRACT

Hugging Face (HF) has established itself as a crucial platform for the development and sharing of machine learning (ML) models. This repository mining study, which delves into more than 380,000 models using data gathered via the HF Hub API, aims to explore the community engagement, evolution, and maintenance around models hosted on HF – aspects that have yet to be comprehensively explored in the literature. We first examine the overall growth and popularity of HF, uncovering trends in ML domains, framework usage, authors grouping and the evolution of tags and datasets used. Through text analysis of model card descriptions, we also seek to identify prevalent themes and insights within the developer community. Our investigation further extends to the maintenance aspects of models, where we evaluate the maintenance status of ML models, classify commit messages into various categories (corrective, performative, and adaptive), analyze the evolution across development stages of commits metrics and introduce a new classification system that estimates the maintenance status of models based on multiple attributes. This study aims to provide valuable insights about ML model maintenance and evolution that could inform future model development strategies on platforms like HF.

CCS CONCEPTS

• **Information systems** → **Data mining**; • **Software and its engineering** → **Software maintenance tools**; *Software libraries and repositories.*

KEYWORDS

repository mining, software evolution, maintenance

ACM Reference Format:

Joel Castaño, Silverio Martínez-Fernández, Xavier Franch, and Justus Bogner. 2024. Analyzing the Evolution and Maintenance of ML Models on Hugging Face. In *21st International Conference on Mining Software Repositories (MSR '24)*, April 15–16, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3643991.3644898>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR '24, April 15–16, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0587-8/24/04...\$15.00
<https://doi.org/10.1145/3643991.3644898>

1 INTRODUCTION

The rapid evolution of machine learning (ML) models, especially on community platforms, is redefining the landscape of AI research and application. Hugging Face (HF) and its Hub [1] stand out in this regard due to their critical role in the development, sharing, and deployment of a wide array of ML models, including Large Language Models (LLMs) and generative AI. HF represents an ecosystem where technical and social dynamics converge, forming a nexus of collaborative development that is continuously evolving. Despite its significance, the understanding of HF's model evolution and maintenance practices remains underexplored.

Previous studies have explored various facets of HF, including pre-trained model reusability [2], the platform's carbon footprint [3], or the challenges in reusing pre-trained models across different domains [4]. Our study aims to provide a holistic view of the current state of ML models on HF, focusing on their evolution, maintenance, and broader implications for the ML community. The novelty of our work lies in the comprehensive examination of these aspects on HF, which, to our knowledge, has not been explored in such detail before.

We delve into the dynamics of ML model maintenance and evolution on HF, investigating domain-specific trends, author collaboration patterns, content evolution in model cards, and detailed maintenance practices. These practices include analyses of commit types, file edits, and maintenance categorization, along with their correlations with model characteristics. Such information can guide users towards actively maintained models and inform their decision-making by highlighting the likelihood of future model updates. Moreover, our findings reveal the unique nature of ML model development compared to traditional software, emphasizing the diverse array of tools used, the crucial role of collaboration, and the distinctive developmental approaches.

The insights garnered from this study are not limited to the HF platform. They offer implications for the maintenance of ML models in general. The patterns and trends identified provide valuable lessons for the broader ML community, regardless of the specific platform, repository or environment used. Therefore, this research aims to guide the development of structured maintenance frameworks, enhancing transparency, and setting community-wide standards for ML model maintenance. Our study paves the way for future research in ML model evolution, offering both a framework and a replication package that can be adapted and applied beyond HF to improve maintenance activities of ML models in general.

2 BACKGROUND AND RELATED WORK

2.1 ML Model Maintenance and Evolution

In recent decades, the rise in data and computing power availability has significantly enhanced ML applications across various domains [5]. ML models, integrated into ML systems [6], require regular maintenance to address *concept drift*—a decline in predictive accuracy over time due to changing data characteristics [7, 8]. Maintenance tasks, crucial yet challenging [9, 10], involve ensuring operational stability and efficiency through corrections, adjustments, and optimizations, aligning with ISO 25059's software maintenance standards [11] and interpretations by Rowe et al. [12].

Contrastingly, evolution in ML models signifies substantial adaptations to new datasets and technologies [13]. As outlined by Bennett and Rajlich [14] and Lehman et al. [15], this encompasses a range of changes across the lifecycle of software, including the introduction of new features and meeting new requirements. Our research separates the overall evolution of the HF community, which involves trends in model development, framework usage, and author dynamics, from the specific maintenance of individual ML models that focuses on routine updates for current functionality.

2.2 The Hugging Face Hub

Training complex ML models requires considerable expertise and resources. Therefore, it is advantageous to reuse existing pretrained models. One community platform to facilitate such sharing and reuse is provided by the company Hugging Face, Inc. (HF). Founded in 2016 as a Natural Language Processing (NLP) company, HF became popular for releasing their NLP models as open source [16] and creating a user-friendly library for NLP transformers [17]. Today, the HF Hub represents their most important product, i.e., a public platform to train, share, download, and deploy ML models and datasets. The Hub adopted the *Model Cards* idea by Mitchell et al. [18]: published models can provide a `README.md` and additional metadata, e.g., tags or prediction quality metrics, which leads to better documentation, transparency, and reproducibility. Models and datasets hosted on the Hub are represented as Git repositories¹, i.e., they are under version control, with multiple people being able to commit changes to them. All in all, the HF Hub is slowly but surely establishing itself as the “GitHub of ML models”.² However, unlike with GitHub, we know little about the state of the HF Hub and how the community uses it.

2.3 Related Work

Two types of publications are related to our study: a) publications about the maintenance and evolution of ML, and b) publications about analyzing the HF Hub. Topic a) has mostly been studied from the perspective of maintainability challenges or technical debt, as visible through the secondary studies by, e.g., Shivashankar and Martini [19] and Bogner et al. [20]. However, some studies also analyzed maintenance and evolution activities in more detail. Tang et al. [21] analyzed 26 open-source ML projects on GitHub and studied how refactoring took place in these repositories to remove technical debt items. Based on their analysis, they conceptualized

new refactorings and technical debt categories specific to ML. They also proposed refactoring-related best practices and antipatterns. Dilhara et al. [22] conducted a mixed-method study to analyze the usage and evolution of ML libraries. They first analyzed over 3,000 open-source repositories containing ML libraries and how their usage evolved. Afterward, they surveyed 109 developers using ML libraries. They identified that ML library updates frequently lead to the update of additional libraries, and that ML libraries are also downgraded in 20% of the cases. Lastly, they highlighted specific challenges for the maintenance and evolution of ML software. To combat the decay of ML models through concept drift, Leest et al. [23] proposed an architectural framework to support making design decisions to prepare an ML-enabled system for evolution. The framework uses scenarios to capture different facets of evolution and to analyze trade-offs between evolvability and other quality attributes. However, the framework has not been empirically evaluated so far, which the authors plan to do via industrial case studies.

Several studies also analyzed different characteristics of the HF Hub. Taking a security perspective, Kathikar et al. [24] analyzed the linked GitHub repositories of 110,000 HF models. They used static analysis to identify a substantial number of vulnerabilities, even though the vast majority were of low severity. However, the share of high-severity vulnerabilities was larger in popular fundamental repositories such as Transformers, which makes securing ML models even more complex. In a previous study of ours [3], where we analyzed around 170,000 models to uncover insights about HF's impact on environmental sustainability, we discovered that only a very tiny percentage of models reported the carbon emissions from their training. Most of these were models trained on the HF infrastructure, which reports these emissions automatically. Over the years, the share of models reporting carbon emissions also decreased, but for those that did report them, mean emissions decreased slightly. We also identified factors correlating with high carbon emissions. Ait et al. [25] wanted to make the analysis of HF more convenient and therefore created *HFCommunity*, a tool that collects and integrates data about the HF Hub, e.g., data on repositories, discussions, files, commits, etc. The data is provided as a relational database dump that can be downloaded and analyzed offline. The authors envision *HFCommunity* as a long-term data source to enable efficient empirical studies of ML projects. Jiang et al. [2] conducted an interview study with practitioners who use HF. They identified practices and challenges regarding the reuse of pretrained models. Afterward, they extended their data with a security risk analysis based on information mined from the HF Hub. They concluded that several risky practices exist in the supply chain of pretrained models, e.g., a frequent lack of signatures. Lastly, Gong et al. [4] explored pre-trained model usage across repositories like HF. They emphasized the need for “model contracts” to address challenges in reusing models due to domain gaps, recommending specifications on intended usage, limitations, and performance for better model reuse.

While several studies have analyzed the maintenance and evolution of ML software, no study reports about these activities for models on the HF Hub. Getting insights into how ML models are maintained and evolve in the largest community platform could lead to the identification of important challenges and practices, and can also inform more design-oriented future research.

¹<https://huggingface.co/docs/hub/repositories>

²<https://www.forbes.com/sites/kenrickcai/2022/05/09/the-2-billion-emoji-hugging-face-wants-to-be-launchpad-for-a-machine-learning-revolution>

3 METHODOLOGY

In this section, we outline our methodology, stating from the study objective and research questions, followed by an explanation of the dataset collection process.

3.1 Study Objective and Research Questions

Following the Goal Question Metric (GQM) guidelines [26], our research goal is structured as follows:

Analyze pre-trained ML models for the purpose of exploring and categorizing with respect to their present status, evolution and maintenance from the viewpoint of ML researchers and practitioners in the context of the HF Hub.

Two main research questions (RQ) arise from this goal. We explore the models in HF to understand their development, popularity, and maintenance:

RQ1. *What is the current status and evolution of the HF community?*

- RQ1.1: How has HF's popularity changed?
- RQ1.2: How have framework usage, tag, and dataset trends evolved in HF?
- RQ1.3: Are there prominent authors groups in HF community?
- RQ1.4: What trends and insights can be identified from the content of HF model cards?

RQ2. *How can we evaluate and categorize the maintenance status of ML models on HF through their commit information?*

- RQ2.1: What do commit metrics reveal about the maintenance of ML models?
- RQ2.2 How does the size and frequency of commits evolve over time?
- RQ2.3: How do different types of commits (perfective, corrective, adaptive) contribute to the maintenance of models?
- RQ2.4 How do the editing patterns of specific files evolve across different development stages?
- RQ2.5: How can we classify the maintenance status of individual models using their commit data?
- RQ2.6: How do various model characteristics differ between maintenance levels?

3.2 Dataset Construction

To answer our RQs, we execute a data collection and preprocessing pipeline, refined to meet the specific demands and objectives of the current study. The data extraction process was carried out on November 6th, 2023.

Data availability statement: The datasets, code, and detailed documentation are available in a replication package hosted on Zenodo [27].

3.2.1 Data Collection. Our data collection pipeline employs the HF Hub API using the `HfApi` class [28], a Python wrapper, to collect data about users and models stored on the HF platform. To this end, we collect a range of common model attributes, including: the total

size of datasets used, hardware used for training, evaluation metrics such as accuracy or F1, size of the model file in the repository, number of downloads and likes for each model, tags attached to each model (e.g., PyTorch, Transformer), the raw text of the model's card and more. For more detailed information on the data attributes and collection process, refer to [27].

In addition to these common attributes, our pipeline is enhanced to collect detailed data related to the commit history of models, providing insights into their development and maintenance over time. This approach is complemented by the integration of data from the *HFCommunity* dataset [25], an offline up-to-date relational database built from the data available at the HF Hub. The *HFCommunity* dataset used the *PyDriller* framework to extract detailed commit information, thereby providing access to the list of files edited in each commit, a feature not available through the HF API. This additional layer of data enriches our analysis by offering a more complete view of the changes made to each model over time.

In addition to commit data, we retrieve discussion data, which includes questions, pull requests, and issues related to the models from the HF API. More details on the data collection are deferred to the replication package.

3.2.2 Data Preprocessing. Our analysis involves the processing of the newly incorporated commit and discussion data. The dataset after the collection phase possesses over 380,000 data entries, each representing a model on HF.

Firstly, we classify commit data to assess the nature of changes made to the models based on its messages. This classification aligns with Swanson's traditional software maintenance categories — Corrective, Perfective, and Adaptive [29]. The classification of commits is performed using a neural network approach based on the work of Sarwar et al. [30], who fine-tuned an off-the-shelf neural network, DistilBERT, for the commit message classification task. We fine-tune the neural network proposed in the paper and use it to classify each of the commits.

Beyond classification, we derive metrics that reflect model evolution and ongoing maintenance efforts, such as the frequency and distribution of various commit types. This process is critical for understanding the lifecycle and robustness of the models.

Moreover, we harmonize variables, manage missing values and identify and handle irrelevant or low-impact attributes appropriately ensuring the dataset's integrity and consistency. The final step in our preprocessing is the application of one-hot encoding to the tags associated with the models. This encoding, combined with a developed tag-to-domain dictionary allows us to filter and map tags to domains, including: Multimodal, Computer Vision, NLP, Audio, and Reinforcement Learning.

3.3 Data Analysis

In this section, we describe the methodology for analyzing the data to answer our research questions. We aim to provide a clear and reproducible account of how we analyzed the data and derived conclusions.

3.3.1 RQ1 Analysis. To address RQ1.1, we construct several time-series graphs using attributes that could indicate and demonstrate an increase in popularity. Specifically, we analyze trends in the

number of new models added each month, the number of commits created, likes, the number of new unique authors, and the number of opened discussions.

For RQ1.2, we analyze the overall statistics of datasets, tags, and libraries, followed by a time-series plot that demonstrates the proportion of the attributes that have been in the top 5 each year. This analysis helps us to identify the trends and popularity of specific tags, datasets, and libraries over time. To assess whether there are statistically significant evolutionary differences over time in the usage of top frameworks (*pytorch*, *tensorflow*, and *jax*), we employ a Chi-squared test for independence. This test is particularly suitable for our analysis, as it allows us to evaluate the relationship between categorical variables (in this case, the frameworks) across multiple time periods.

To address RQ1.3, we employ a graph-based approach to uncover groups of authors using the Louvain algorithm [31]. We choose the Louvain method for its efficiency and effectiveness detecting communities in a large network such as HF. We construct a graph with authors as nodes and collaborations as edges, attributing model popularity to each author and linking co-authors. We define the popularity on each author as the sum of the popularity of each model they collaborated on. The Louvain method was then employed for community detection, identifying clusters of closely connected authors (i.e., author groups). Subsequently, we calculated the cumulative popularity of each author group, ranked them in descending order, and visualized the concentration of popularity among the top groups.

Lastly, for RQ1.4, we use Latent Dirichlet Allocation (LDA) [32] to identify common topics and their evolution within the text of the model cards. For LDA's hyperparameters tuning, we conduct experiments with topic coherence metrics, such as C_v [33], and perform manual inspection of the topics to ensure they are distinct and meaningful. This approach is complemented by testing various hyperparameters, including document and word topic densities.

3.3.2 RQ2 Analysis. For the maintenance status of models, we analyze the descriptive statistics of five main maintenance metrics (RQ2.1): the number of commits per model, average number of files edited by the commits for a model, monthly commit frequency, and the total number of authors involved in a commit, as in HF a single commit for a model can be made by multiple authors.

For RQ2.2, we examine the evolutionary trend of the number of commits and commit size, employing a slope t-test on a fitted linear regression with a significance level of $\alpha = 0.05$ to test for any significant trends.

For RQ2.3, we use the classified maintenance types (perfective, corrective, and adaptive) to analyze their proportions, and how they evolve throughout a model's development lifecycle. That is, we calculate the proportion of commit types across development stages (from beginning to end) and plot the evolution of these proportions throughout the development process.

For RQ2.4, we identify the most commonly edited files in commits and analyze the file editing lifecycle to observe how the patterns of these edits change over the course of a model's development. Equivalently with RQ2.3, we measure the proportion of edits to specific files at five key stages in the development cycle, providing insights into the evolving nature of file modifications as the model

matures. Finally, to better understand the relationships between files commonly edited together, we construct a graph where nodes represent individual files, and edges denote the co-occurrence of file edits within the same commit. The weight of each edge corresponds to the frequency of these co-editing events, offering a quantitative measure of the strength of the relationship between files. Using again the Louvain algorithm, we then detect communities within this graph. These communities are groups of files that are frequently edited together, which we describe and visualize.

For RQ2.5, we employ a k-means clustering algorithm to classify the maintenance status of ML models on the HF platform. We opt for $k = 2$ based on initial observations of minimal variance for $k > 2$, ensuring a distinction between high and low maintenance models, aligning with similar approaches such as Coelho et al. [34]. K-means is selected for its simplicity and effectiveness in generating distinct, interpretable clusters, ideal for delineating clear maintenance groups. In contrast, methods like DBSCAN [35], which automatically determines the number of clusters based on data density, might not explicitly align with our specific objective of categorizing into two maintenance categories. The classification is based on several key features: the total number of commits, the frequency of commits per month, the average interval between commits, the longest duration without any commits, the count of contributing authors, and the proportion of discussions that have been successfully closed. The selection of these features was influenced by their relevance to maintenance activities, and was also informed by similar attributes used in parallel research [34]. Details can be further seen in the replication package.

Finally, for RQ2.6, we investigate how various model characteristics differ between high and low maintenance categories. Our analysis encompasses both continuous and nominal variables, using statistical tests appropriate for non-normally distributed data at significance level $\alpha = 0.05$. For continuous variables, including likes, downloads, size, model card text length, accuracy, f1, and datasets size, we employ the Mann-Whitney U test [36]. This test is suitable for comparing the means of two independent samples without assuming normality. For nominal variables, specifically the domain and library usage, we use the Z-test for proportions. This test allows us to determine whether the proportion of models in a particular domain or using a specific library significantly differs between the high and low maintenance categories.

4 RESULTS

In this section, we present the results of the data analysis per RQ.

4.1 Current Status and Evolution of HF Community (RQ1)

4.1.1 How has HF's popularity changed? The results displayed in Figure 1 clearly showcase HF's exponential growth in popularity.

The analysis of various metrics from 2017 to 2023 on the HF platform reveals a consistent and significant uptrend in engagement and development activities. Notably, there has been a clear increase in the number of new models added each month. This trend is accompanied by a marked spike in community engagement, as evidenced by the rise in likes and discussions, particularly from 2022

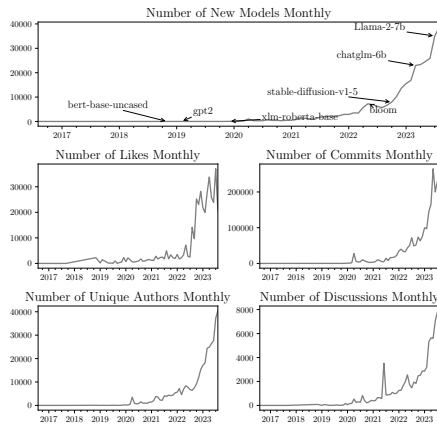


Figure 1: Popularity metrics evolution on HF.

onwards. Additionally, there has been a steady growth in both the number of commits and the diversity of contributing authors each month. These trends collectively highlight the growing importance and popularity of HF in the ML community, aligning with findings from previous studies [3].

Finding 1.1. HF's popularity has exponentially increased over time, which is evident from the upward trends in the number of new models, likes, commits, unique authors, and discussions aggregated monthly.

4.1.2 How have framework usage, tags and datasets trends evolved in HF? Regarding the most common libraries, *transformers* and *pytorch* hold the first and second positions respectively, with *transformers* being used in 163,936 models and *pytorch* in 150,757 models. This is further supported by the top frameworks on each domain, where *transformers* and *pytorch* appear across domains such as Audio, Computer Vision, and NLP. For Multimodal models, the top library is *diffusers*, which has state-of-the-art pretrained diffusion models for generating images and audio. For Reinforcement Learning tasks, we have *Stable-Baselines3*, a set of implementations of reinforcement learning algorithms built on top of PyTorch.

In total, there are 129 unique libraries used across various models on HF. The evolution of the relative popularity of top libraries can be seen in Figure 2. This figure is a line chart showing the trends of the proportion of various libraries from the fourth quarter of 2018 to the fourth quarter of 2023. It is important to note that a single model can incorporate multiple libraries, resulting in several libraries having high proportions (e.g., *tensorflow*, *jax* and *safetensors* at 2019Q1). Furthermore, we recognize that certain libraries like the *transformers* (HF Transformers library [37]) may be used in conjunction with others such as *pytorch* or *tensorflow*. In our analysis, these are only counted if they are explicitly mentioned to prevent false positives. The following observations can be made:

- **Dominant Libraries:** *pytorch* and *transformers* are consistently the most dominant tags, although the total proportion of models with these tags has been shrinking until reaching

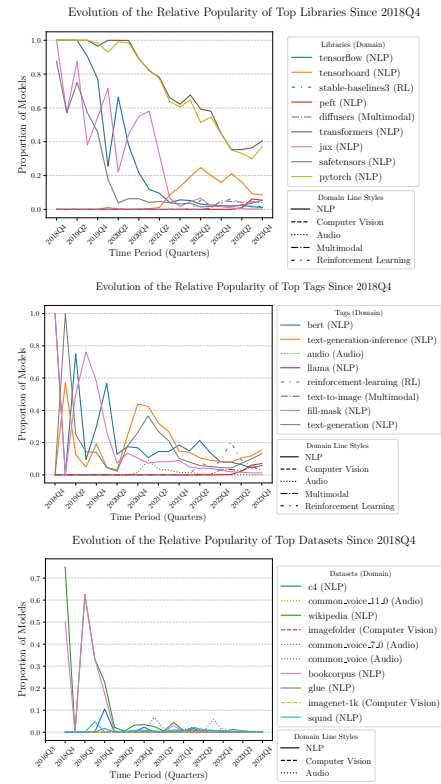


Figure 2: Evolution of the relative popularity

≈ 40% today, suggesting an increased variety of tags. The observed decline reflects the increasing diversity of models on HF. As the platform grew, a wider array of models emerged, leading to a dilution in the proportion of models using these libraries.

- **Reinforcement Learning Libraries:** *stable-baselines3* experienced a surge in popularity at the beginning of 2022, but it remains less prominent compared to major NLP libraries.
- **Multimodal Libraries:** *diffusers* experienced a surge in popularity in mid-2022, ranking as the fifth most popular library in the last quarter.
- **Library Comparison:** In comparing popular frameworks, *pytorch* remains the most popular, while *tensorflow* and *jax* have seen notable decreases in usage. Specifically, *pytorch*'s usage declined by 62.79%, whereas *tensorflow* and *jax* experienced sharper declines of 98.85% and 99.85%, respectively. The Chi-square test confirms these trends are statistically significant ($p\text{-value} < 0.05$).

Finding 1.2.1. 'transformers' and 'pytorch' are the most used frameworks overall. Moreover, 'pytorch' maintained the framework dominance while 'tensorflow' and 'jax' lost popularity.

As we delve further into the evolution of HF, we observe a shift in the popularity of tags and datasets over the years. As for the

tags, there are a total of 23,496 unique tags on HF. The most frequent tags encompass a range of topics, from library-specific and auxiliary tags to language-related tags. However, when we filter out these non-specific tags, we uncover the true interests of the HF community. Generative AI and NLP-related tags such as *text-generation-inference*, *text-classification*, and *reinforcement-learning* are particularly prevalent.

Figure 2 also illustrates the dynamic landscape of tag popularity over the years. We observe a steady decline in older NLP models like BERT, while tags related to generative AI have gained momentum. Although there is a consistent interest in audio-related tasks and reinforcement learning, they remain less popular than some of the NLP tags. The last three quarters have witnessed a surge in interest for text-generation-inference, text-to-image, and llama.

Finding 1.2.2 *The analysis of tags reveals a dominant interest in generative AI and NLP within the HF community, with notable but less significant interest in other domain tags such as audio-related tasks and reinforcement learning.*

Finally, in terms of datasets, a total of 10,525 unique datasets are used. In Figure 2, we can also observe the changing popularity of datasets over time. In the earlier years, NLP datasets like GLUE and Wikipedia were foundational, serving as benchmark datasets. However, their popularity has waned over time, possibly due to the emergence of newer datasets or shifting research priorities, without new hugely dominant datasets on the community.

Finding 1.2.3. *Although NLP datasets like GLUE and Wikipedia were foundational in earlier years, there are no dominant datasets nowadays.*

4.1.3 Who are the prominent authors and what are their relationships in HF? An analysis of the most common authors reveals names such as 'SFconvertbot' and 'librarian-bot', which are automated bots contributing to the platform among other human authors.

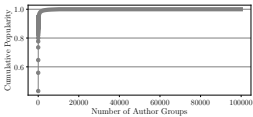


Figure 3: Cumulative popularity of author groups

Metric	Value
Average Downloads	1,858,571
Average Likes	294.71
Average # of Authors	8.61
Average Length of Card	8,807.55

Table 1: Average Statistics Top Author Group

To further explore the relationships between authors, we employed the Louvain algorithm to identify clusters of authors who frequently collaborate with each other. The results were quite telling. A small number of groups garnered the majority of popularity on HF, with the first group alone accounting for 40% of the platform's popularity as can be seen in Figure 3. This is further evidenced in Table 1, which displays statistics for the top author group models that significantly surpass the average metrics of a typical HF model. For example, while the average number of likes for a model is 1.13, the top group boasts an average of 294.71. This group consisted of approximately 580 authors.

The dominance of this group is further emphasized when considering that HF has over 100,000 unique authors. This means that a tiny fraction of authors (approximately 0.5%) are responsible for a significant portion of the platform's popularity. The collaborative nature of these authors is evident in their contributions to models with extensive collaboration such as bigscience/bloom with 22 unique collaborators, among others.

These findings reveal a clear concentration of popularity among a small number of authors who tend to collaborate frequently, indicating a tight-knit community of contributors who play a significant role in shaping the landscape of HF.

Finding 1.3. *A small number of author groups, particularly one dominant group, convey the majority of popularity in HF. This indicates a concentrated popularity among authors who often collaborate with each other.*

4.1.4 What trends and insights can be identified from the content of model cards? An LDA decomposition was conducted to categorize the prevalent themes within the model card content on all published model cards on HF (87,775). We chose symmetric Dirichlet priors, assigning equal prior weight to each topic, as this yielded similar results across multiple choices and ensured a balanced representation of topics. With five components, we ensured that the topics generated were distinct and meaningful. The identified raw topics (top 10 words for each topic) were:

- **Topic 1: Training Info**
 - *Raw*: "training information needed model loss hyperparameters evaluation results following"
 - *Interp.*: About model training specifics.
- **Topic 2: Text Generation**
 - *Raw*: "model huggingface llama 7b information use needed 13b prompt models"
 - *Interp.*: Linked to text generation, with references to llama, prompting or common number of parameters.
- **Topic 3: Reinforcement Learning**
 - *Raw*: "agent model td playing baselines3 false python stable github rl"
 - *Interp.*: Models centered on agent-based learning.
- **Topic 4: NSFW Content**
 - *Raw*: "png previews click nsfw style f1 strong font suit maid"
 - *Interp.*: Generation of explicit adult content via HF's NSFW mainly with text-to-image generative content.
- **Topic 5: Other**
 - *Raw*: "model huggingface main github import image resolve use trained models"
 - *Interp.*: Includes miscellaneous and uncategorized cards.

As observed, our topics do not align perfectly with the categories presented by Mitchell et al. [18], suggesting unique trends and focuses in the HF community to explore further.

The evolution of these topics is presented in Figure 4. A growing popularity of NSFW content since the mid 2022 aligns with the urge in popularity of image diffusion models. Simultaneously, the consistent rise in text generation, also propelled by the generative AI wave, emphasizes its lasting significance. "Training Information" dominates the model card topics, indicating that model-specific

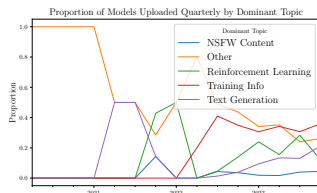


Figure 4: Time series analysis of model cards LDA topics

training details remain in model cards. Lastly, the cyclical nature of "Reinforcement Learning", echoing similar patterns in tag evolution, highlights periodic surges in interest, potentially aligned with advancements or novel applications in this field.

Finding 1.4. Model cards combine technical terms, training parameters, and general descriptors, indicative of the nature their content. Emerging trends like generative AI underscore evolving user interests and applications.

4.2 Maintenance Analysis (RQ2)

4.2.1 What do commit patterns and classifications reveal about the diversity in model development on HF? The diversity in development activities among HF models is clearly evidenced by the range in the number of commits (Figure 5). The average number of commits is 7.16, but the median is only 3.0, showing a strong skew in the distribution with a mean substantially higher than the median. This discrepancy is also reflected in the top models by the number of commits. A few models, such as `CivitAI_model-info` with 97,237 commits, have undergone extensive revisions and updates. In contrast, a significant number of models have very few commits (e.g., 71% of the models have less than 5 commits), suggesting disparities in active development or maintenance. The abnormally large numbers of commits on some models are mostly attributed to automatic commits using the HF API, e.g., updating the model file with a commit on every training step, thereby giving an illusion of high maintenance.

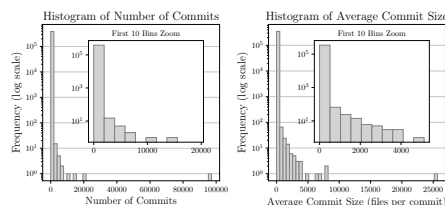


Figure 5: # of commits and commit size per model histograms

Finding 2.1.1. HF is characterized by a diverse but right-skewed distribution of commit patterns (influenced by automated processes), with few models receiving extensive updates and the majority showing limited activity.

The commit size and frequency further highlights the diversity in development practices (Figure 5). While the average commit size is 5.0 files per commit, the median is 2.0, indicating that most changes

are incremental and minor, but there are occasional substantial modifications that could represent significant feature additions or improvements to a model. The frequency of commits also exhibit a wide range, with some models being updated frequently, while others have long intervals between commits: the mean time between commits is 52.6 days, while the median is 27 minutes.

Finding 2.1.2. Incremental improvements dominate model development, as evidenced by the prevalence of minor commits.

When we look at the 100 most popular models, we can identify models with high maintenance metric values from actual development processes rather than automated commits. For instance, the top models by number of commits are `OrangeMixs` with 185 commits, `bloom` with 108 commits, and `chatglm-6b` with 95 commits. Similarly, models such as `lilyasviel/ControlNet-v1-1` with an average commit size of 5.66 files per commit showcase substantial modifications indicative of significant developmental efforts.

The involvement of diverse contributors in model development on HF ranges from individual developers to collaborative efforts. While the majority of models are developed by a small number of authors (with a median of 1 author per model), there are noteworthy exceptions. Models like `bigscience/bloom` or `bigcode/santacoder` are examples of collaborations that bring 22 and 17 unique authors respectively.

Finding 2.1.3. HF encompasses both individual and collaborative efforts. While the average number of unique authors per model is low, (1.18 mean and 1.0 median), there are notable examples of collaboration.

4.2.2 How does the size and frequency of commits evolve over time? Figure 6 presents the quarterly evolution of the number of commits and the average commit size for all models.

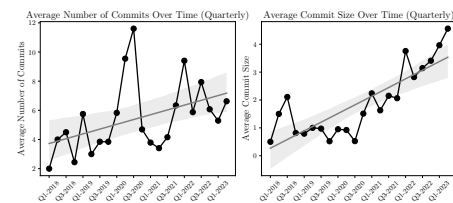


Figure 6: Average number of commits for each model quarterly

As observed in Figure 6, there is a slight upward trend in both the average number of commits and the average commit size over time. This suggests that there might be an increase in the maintenance efforts put into the models hosted on the HF platform. This increase can be attributed the increasing awareness of the importance of model maintenance, an overall increase in the quality and complexity of the hosted models, or a more distributed usage of the HF API, which makes automatic commits easier. The p-value on the slope t-test on both trends is < 0.05 , confirming that this is not just a random fluctuation, but a significant statistical trend. Furthermore, the peak observed in the number of commits and the corresponding decrease in the average commit size during 2020-Q3

can be attributed to the 'Helsinki-NLP' organization. They uploaded a substantial number of models (over 300), characterized by a high frequency of commits and a low number of files edited per commit.

Finding 2.2. *There has been a slight but statistically significant increase in the average number of commits and the average commit size over time, indicating a possible increase in model maintenance efforts on the HF platform.*

4.2.3 How do different types of commits (perfective, corrective, adaptive) contribute to the maintenance of models on the HF platform? The classification algorithm [30] provided multiple labels, including combinations between perfective, corrective, and adaptive. We present examples for each commit type:

- **Corrective Commits:**
 - 'Updated bug in TensorFlow usage code (README.md) (#5)'
 - '[FIX] Fix Typo (#3)'
- **Perfective Commits:**
 - 'For clarity, delete deprecated modelcard.json'
 - 'Update tokenizer.json'
- **Adaptive Commits:**
 - 'Adding 'safetensors' variant of this model (#1)'
 - 'Add size details'

As we can observe, corrective commits address bugs and errors, such as fixing typos or updating incorrect code. Perfective commits focus on improvements and refinements, like updating a tokenizer or deleting deprecated files. Lastly, adaptive commits add new features or variants.

By analyzing the commit type frequencies of 2,760,224 commits, we found a dominant proportion of perfective commits, as shown in Table 2.

Considering *Finding 2.1.2*, it is reasonable to infer that the majority of the commits are perfective in nature. The high proportion of perfective commits aligns with the trend of incremental small improvements being dominant in model development. In fact, significant portion of perfective commits corresponds to routine updates such as 'update README.md', 'update pytorch_model.bin', and other similar enhancements.

Finding 2.3.1. *Perfective commits constitute the majority of maintenance activities on HF. This suggests a strong emphasis on incremental improvements and routine updates during the model development lifecycle.*

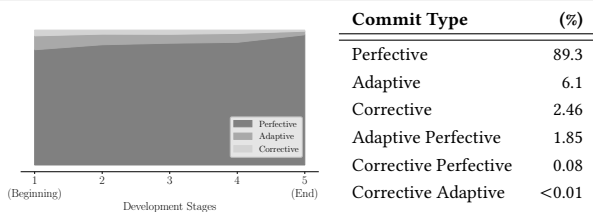


Figure 7: Lifecycle of commit types

Table 2: Proportions of commit types

Further, we sought to understand the lifecycle of commit types over the development of an average model. The results, depicted in Figure 7, which shows the proportion of different types of commits across various stages of development, indicates a noticeable trend. From the graph, we observe the following:

- Perfective commit are dominant at the start and increase even more towards the end development stage.
- Adaptive commits decrease as development progresses.
- Corrective commits maintain a consistent, thin layer throughout the stages with a slight decrease at the end.

The chart suggests that model development on the HF platform typically begins with a mix of perfective and adaptive tasks, gradually transitioning towards perfective efforts, which dominate towards the end. This implies that, as the development matures, there are fewer environmental or requirement changes (adaptive) and a stable number of bug fixes or issue resolutions (corrective), with an increasing focus on enhancing existing features (perfective).

Finding 2.3.2. *Throughout the lifecycle of a model, there is an increase on perfective tasks. This indicates a maturing of model development where enhancements take precedence over new features or the rectification of defects.*

4.2.4 How do the editing patterns of specific files evolve across different development stages? An analysis of the most commonly edited files reveals `pytorch_model.bin` as the most edited filename followed by `README.md` and `.gitattributes`. The lifecycle of files edited in commits, depicted in Figure 8, provides further insights. From the figure, we can draw several conclusions:

- The editing proportion of `pytorch_model.bin` decreases over development stages, indicating its core component status and decreasing need for modifications.
- A significant spike in edits for `.gitattributes` at stage 5.
- A slow and steady drop in edits for `README.md`.
- An increase in edits to `config.json` during the middle stages of development, followed by a gradual decline.
- Files such as `special_tokens_map.json` and `tokenizer_config.json` experience a marginal number of edits throughout the entire development stage.

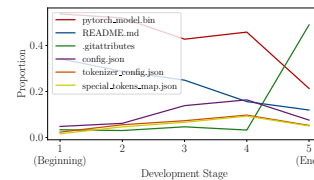


Figure 8: Lifecycle of files edited in commits

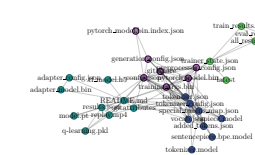


Figure 9: Files Network Graph Clustering

Therefore, the development lifecycle of the typical HF model is characterized by an initial phase of active development and fine-tuning, followed by a stabilization phase where fewer changes are needed. This reflects the iterative process of model development, where the initial focus is on getting the architecture and weights

right, followed by optimization and fine-tuning, and finally documentation and other supporting files. Additionally, Figure 9 illustrates the clustering derived from the Louvain algorithm applied to a graph representing files that are commonly edited at the same time (for a higher resolution version, refer to the replication package). This analysis identified four primary clusters: tokenizer-related files (e.g., `tokenizer.json`, `tokenizer_config.json`), model and configuration files (such as `pytorch_model.bin`, `training_args.bin`), training results data (including examples like `train_results.json`, `eval_results.json`), and a miscellaneous cluster featuring files such as `README.md` and `.gitattributes`. These clusters represent groupings of files that are frequently edited together at the same time.

Finding 2.4.1. *Reduced edits in `pytorch_model.bin` indicate a shift from initial development to stability, with changes in `README.md` or `config.json` marking the transition from setup to final tuning in model development.*

Finding 2.4.2. *The clustering analysis identifies file clusters that represent files frequently edited concurrently (tokenizer, model, training results, and miscellaneous files) highlighting synchronized editing patterns and interdependencies.*

4.2.5 Classification of Model Maintenance Using Commit Data. Our k-means clustering algorithm segregated the ML models into distinct maintenance categories based on their activity patterns in the repository, resulting in two primary categories:

- **High Maintenance Category:** Models with active maintenance practices, characterized by a higher number of commits, regular commit frequency, shorter intervals between commits, fewer days without commits, and a slightly higher number of authors.
- **Low Maintenance Category:** Models with less frequent maintenance activities, indicated by fewer commits, lower frequency of commits, longer intervals between commits, more days without commits, and fewer authors involved.

With 62,818 models classified as high maintenance and 319,477 models as low maintenance (83.5% vs 16.4% respectively) the classification underscores the diverse nature of model maintenance within the HF ecosystem. An analysis of the centroids for both clusters provides quantitative insights into the maintenance behaviors:

Category	Num Commits	Commit Frequency	Avg Days Between Commits	Max Days Without Commits	Num Authors	% Closed Discussions
High Maintenance	28.7	8.0	8.0	170.7	1.5	17.3
Low Maintenance	3.0	0.6	61.2	255.9	1.1	0.1

Table 3: Mean Centroids of Maintenance Categories

The mean centroids clearly illustrate the distinction between high and low maintenance models, with high maintenance models showing greater engagement and activity. In Figure 10, the biplot derived from a Principal Component Analysis (PCA) visually represents the separation between these two maintenance categories. This figure effectively represents the division of models into high and low maintenance categories, with each dot representing a model, and the positioning informed by the maintenance attributes.

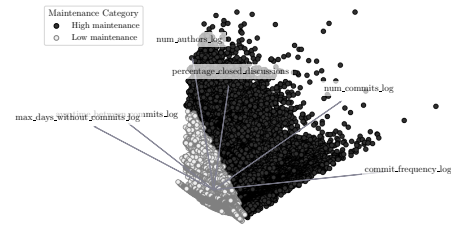


Figure 10: Biplot of K-Means Clustering on Maintenance Features

Finding 2.5. *K-means clustering categorized 16.4% of HF models as 'High Maintenance' and 83.5% as 'Low Maintenance', underscoring diverse maintenance practices and clear distinctions in engagement, as reflected in the centroids.*

4.2.6 How do various model characteristics differ between maintenance levels? Our analysis revealed significant differences in several model characteristics between high and low maintenance categories. We summarize the findings in two tables: one for continuous variables analyzed using the Mann-Whitney U test and another for nominal variables using the Z-test for proportions.

Variable	High Maintenance Mean	Low Maintenance Mean	p-value
Popularity	0.00073	0.000029	<0.001
Likes	5.51	0.25	<0.001
Downloads	11,390.46	241.04	<0.001
Size (MB)	5,976,630.0	1,308,191.6	<0.001
Model Card Text Length	4,132.9	2,070.9	<0.001
Accuracy	0.8002	0.8202	0.13
F1 Score	0.754	0.7370	0.748
Dataset Size (MB)	48,672,083.7	33,105,320.8	0.12

Table 4: Mann-Whitney U Test Results for Continuous Variables

Variable	High Maintenance Proportion	Low Maintenance Proportion	p-value
NLP	0.7303	0.6248	<0.001
Audio	0.0987	0.0798	<0.001
Computer Vision	0.0684	0.0499	<0.001
Multimodal	0.0886	0.0670	<0.001
Reinforcement Learning	0.0140	0.1784	<0.001
tf	0.0552	0.0222	<0.001
jax	0.0319	0.0208	<0.001
transformers	0.6965	0.3830	<0.001
pytorch	0.6473	0.3414	<0.001

Table 5: Z-Test for Proportions Results for Nominal Variables

The results indicate that models with high maintenance tend to be more popular, have more likes and downloads, and are larger in size compared to those with low maintenance. In fact, when we select the leading author group identified in RQ1.3 and classify the top 500 models that have the most authors in common with this group, we find that 98% of these models fall into the high maintenance category. This suggests that the concentration of popular author groups also extends to a concentration of high maintenance activities within specific author groups. Additionally, the model card text length is significantly longer for high maintenance models, suggesting more extensive documentation in this category.

In terms of nominal variables, the domains of NLP, Audio, Computer Vision, and Multimodal showed a higher proportion for high

maintenance, whereas Reinforcement Learning was more prevalent for low maintenance. For libraries, 'transformers' and 'pytorch' were predominantly used in high maintenance models, whereas the others showed significant but less pronounced differences.

Finding 2.6. *High-maintenance models tend to be more popular, larger, and better documented than their low-maintenance counterparts, with a notable concentration of high maintenance activities within specific author groups.*

5 IMPLICATIONS

This study offers a comprehensive examination of the evolution and maintenance practices within the HF community, providing significant insights that can spearhead advancements in the ML domain. These have relevant implications for both researchers and practitioners, providing them with a deeper understanding of the dynamics in model evolution that can inform best practices for model maintenance and evolution in community-driven platforms.

5.1 Status and Evolution of Hugging Face (RQ1)

The evolutionary insights presented significant trends in model development on HF, offering a predictive lens for the future trajectory of ML research and applications. Our findings chart the progressive details of model evolution, providing a valuable barometer for the ML community's direction.

- **Predictive Trends for Strategic Alignment:** By mapping the growth patterns in model additions and framework usage (Finding 1.1 and 1.2), we provide a predictive foundation for researchers and developers to strategically align their efforts with future demands and community directions.
- **Emphasis on Collaboration:** The insights into authorship dynamics (Finding 1.3 and Finding 2.6) indicate that high-maintenance models, which are more popular and better documented, often emerge from these collaborative multi-author environments, emphasizing the impact of collective efforts on model quality and visibility.
- **Model Documentation as a Reflective Mirror:** The growing emphasis on generative AI in model cards, as seen in Finding 1.4, underscores the dynamic development of ML and the need for robust documentation. This evolving landscape underscores the necessity for robust documentation, echoing Oreamuno et al. [38]'s observation of inadequate documentation in many HF models and datasets. This issue is compounded by Finding 2.5's revelation of prevalent low maintenance in models, aligning with Bhat et al. [39]'s call for responsible, up-to-date documentation practices. As ML models evolve in complexity, it is imperative that their documentation maintains high standards of clarity, completeness, and ethical considerations, enhancing accountability and usability across applications.

5.2 Maintenance and Evolution of Models (RQ2)

Our analysis reveals significant variance in maintenance practices across HF models, underscoring the need for systematic, collaborative, and continuously refined maintenance approaches.

- **Understanding File Edit Interdependencies:** Finding 2.4.2 reveals how clustering analysis can highlight synchronized editing patterns and interdependencies among file types. This knowledge is valuable for anticipating and managing linked changes in model files, which can streamline maintenance processes and minimize errors. Furthermore, if HF facilitated the retrieval of line-level change data through its API, the analysis of file interdependencies could be greatly enhanced. Zimmermann et al. [40] have demonstrated the importance of mining version archives at the line level. This granular approach could provide deeper insights into ML model evolution.
- **Lifecycle Planning:** Understanding the typical lifecycle of model development (Finding 2.4.1) offers practical benefits for developers in optimizing their maintenance strategies. For instance, the transition from frequent edits in `pytorch_model.bin` to adjustments in `README.md` or `config.json` can be used as indicators to recognize when a model is shifting from its development phase to stabilization and refinement. This awareness enables developers to allocate resources more efficiently, ensuring that the development resources are used at the right stages of the model's lifecycle, leading to more efficient and effective model evolution.
- **Refined Maintenance Categorization:** The implementation of the maintenance classification, as revealed in Finding 2.5, not only enables users to make more informed choices by identifying models that are actively maintained but also introduces the potential for a Long Term Support (LTS) model in HF. This approach would categorize certain models as LTS, indicating a commitment to longer-term stability, regular updates, and support, enhancing transparency and ensuring that users can rely on these models for extended periods without significant changes disrupting their projects. For developers, this structured framework provides a clear roadmap for prioritizing maintenance tasks.

5.3 ML Systems vs. Traditional Repositories

The maintenance of ML models on HF presents a unique pattern compared to the maintenance of traditional software systems. In the realm of traditional software system development, such as those found in repositories like GitHub, the focus typically lies on bug fixes and feature additions. This involves version releases and systematic testing cycles [41], reflecting a development paradigm where changes are often driven by evolving user requirements or efforts at software optimization.

In HF's ML model development, we noted individual and collaborative efforts, highlighted by the varied nature of maintenance activities. This variance is reflected in the distribution of commit patterns, where perfective maintenance emerges as the dominant approach (Finding 2.3). Such an approach, focusing on incremental model improvements, contrasts with traditional software system development seen in repositories like GitHub. In the HF context, the maintenance of ML models prioritizes enhancing model performance and aligning with evolving technological advancements.

This trend indicates a departure from the traditional software maintenance paradigms. It reveals the need for methods and tools

specifically designed for the unique demands of ML model maintenance. Such tools may include advanced version control systems optimized for data and model tracking, as well as automated monitoring tools capable of detecting model drift or degradation. These tools and methodologies should align with the principles of continuous learning, model monitoring, and dynamic adaptation to data changes, which are crucial for maintaining the quality and relevance of ML models over time. The exploration of MLOps tools such as DVC [42] and DagsHub [43], as discussed in Lanubile et al. [44], showcases the potential in this area.

The implications of these insights extend beyond the HF community, affecting the broader field of ML. They underscore the necessity for a paradigm shift in how ML models are conceptualized and maintained, potentially enhancing the efficiency, reliability, and overall effectiveness of ML development in community-driven platforms like HF.

6 THREATS TO VALIDITY

In this section, we discuss the potential threats to the validity of our study and outline the mitigating actions we have taken to minimize these threats.

Construct Validity: Although we have employed comprehensive data collection and preprocessing methodologies, there is a possibility that the data may contain inaccuracies, inconsistencies, or missing values that could affect the results. This situation is exacerbated by the absence of standardized reporting for metadata on ML models. Moreover, to effectively measure constructs like popularity, maintenance, and evolution, we use relevant indicators and metrics. However, these might not fully capture the constructs' complexity, indicating a need for further research and refinement.

Mitigation: We have implemented rigorous data cleaning and preprocessing procedures. We have also cross-validated the data obtained from the HF API with the *HFCommunity* dataset to ensure consistency and completeness. For future studies, the implementation of model metadata extractors (e.g., [45]) could be considered to enhance data quality further.

Internal Validity: Our classification of commits into corrective, perfective, and adaptive types is based on a neural network approach, which may introduce bias due to the training data or model architecture used.

Mitigation: To mitigate this threat, we used a proven methodology from previous research and performed a validation check to ensure the accuracy and reliability of the commit classification. Sarwar et al. [30] reports a test accuracy of 89%. We further manually analyzed 125 commit messages along its classifications to check the alignment of the results achieving 86% accuracy.

External Validity: Our study is based on data collected from the HF platform, which may limit the generalizability of our findings to other ML model platforms or communities. Additionally, our study focuses on the HF models as of November 6, 2023, and the findings may not be applicable to future developments on the platform.

Mitigation: Our methodology is robust and replicable, designed to be applied to future datasets or similar platforms. We have provided a detailed methodology and a replication package to enable validation of our findings with new data, ensuring the broader applicability and relevance of our approach.

Reliability: Our study relies on a reproducible research methodology, where the data collection, preprocessing, and analysis procedures are clearly outlined. However, there is a possibility that changes in the HF API or *HFCommunity* dataset structure could affect the reproducibility of our study.

7 CONCLUSIONS

This study presented a detailed examination of the evolution and maintenance of ML models on the HF platform, with a focus on two central research questions. The results offer a detailed understanding of the dynamics shaping model development and underscore the importance of systematic maintenance and incremental improvement for long-term model efficacy.

We observed that the HF community is not only expanding in terms of model quantity but also evolving through the adoption of new frameworks and tags, reflecting shifts in focus and innovation within the ML landscape, especially in the realm of generative AI. The analysis unveiled a vibrant ecosystem where certain models and tags gain prominence, indicative of the community's responsiveness to emerging trends and challenges in the field.

Moreover, our findings revealed that, while model development on HF encompasses both individual and collaborative efforts, there is a significant variance in the activity of model maintenance, as seen in the diverse distribution of commit patterns. Notably, perfective commits dominate, suggesting a continued focus on refining and optimizing models. The lifecycle of commits and the editing patterns of specific files further highlight different phases of model development, with an initial active development stage that transitions into stability and efficiency optimization in core components.

Additionally, we propose a framework for classifying models by their maintenance status, which could be instrumental for users in selecting models that align with their reliability and support requirements. Encouraging transparency in maintenance logs is essential in fostering a trust-based relationship between developers and the community. Moreover, the study highlights the unique maintenance dynamics of ML models on HF, diverging from traditional software paradigms with a focus on perfective maintenance, needing tools and methods tailored to ML's evolving needs.

For future work, developing advanced tools for automated and predictive maintenance to proactively address potential model issues is one crucial area. Investigating the social dynamics of model development can shed light on collaborative patterns, author roles, and best practices within the community. Extending this study to other ML platforms and comparing maintenance practices will provide insights into the broader ML development landscape.

In closing, this paper calls for concerted efforts towards analyzing the growth trajectory of ML model repositories while emphasizing the criticality of maintenance practices. We urge for enhanced transparency, structured maintenance frameworks, and community-wide standards that will propel the ML community towards greater heights of excellence and innovation.

ACKNOWLEDGMENTS

This work is supported by the project TED2021-130923B-I00, funded by MCIN/AEI/10.13039/501100011033 and the European Union Next Generation EU/PRTR.

REFERENCES

- [1] Hugging Face Inc., “Hugging Face Hub Documentation,” <https://huggingface.co/docs/hub/index>, 2023.
- [2] W. Jiang, N. Synovic, M. Hyatt, T. R. Schorlemmer, R. Sethi, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, “An Empirical Study of Pre-Trained Model Reuse in the Hugging Face Deep Learning Model Registry,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia: IEEE, May 2023, pp. 2463–2475. [Online]. Available: <https://ieeexplore.ieee.org/document/10172757/>
- [3] J. Castaño, S. Martínez-Fernández, X. Franch, and J. Bogner, “Exploring the Carbon Footprint of Hugging Face’s ML Models: A Repository Mining Study,” in *ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. New Orleans, LA, USA: IEEE, 2023.
- [4] L. Gong, J. Zhang, M. Wei, H. Zhang, and Z. Huang, “What is the intended usage context of this model? an exploratory study of pre-trained models on various model repositories,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, pp. 1–57, 2023.
- [5] I. H. Sarker, “Machine Learning: Algorithms, Real-World Applications and Research Directions,” *SN Computer Science*, vol. 2, no. 3, p. 160, May 2021. [Online]. Available: <https://link.springer.com/10.1007/s42979-021-00592-x>
- [6] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, “Software Engineering for AI-Based Systems: A Survey,” *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 2, pp. 1–59, Apr. 2022. [Online]. Available: <https://dl.acm.org/doi/10.1145/3487043>
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under Concept Drift: A Review,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 1–1, 2018, eprint: 2004.05785. [Online]. Available: <https://ieeexplore.ieee.org/document/8496795/>
- [8] J. L. Leevy, T. M. Khoshgoftaar, R. A. Bauder, and N. Seliya, “Investigating the relationship between time and predictive model maintenance,” *Journal of Big Data*, vol. 7, no. 1, p. 36, Dec. 2020. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00312-x>
- [9] A. Paleyes, R.-G. Urma, and N. D. Lawrence, “Challenges in Deploying Machine Learning: A Survey of Case Studies,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, Jul. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3533378>
- [10] R. Nazir, A. Bucaioni, and P. Pelliccione, “Architecting ML-enabled systems: Challenges, best practices, and design decisions,” *Journal of Systems and Software*, vol. 207, p. 111860, Jan. 2024. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0164121223002558>
- [11] ISO/IEC 25010, *ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, Std., 2011.
- [12] D. Rowe, J. Leaney, and D. Lowe, “Defining systems evolvability—a taxonomy of change,” *Change*, vol. 94, pp. 541–545, 1994.
- [13] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software Engineering for Machine Learning: A Case Study,” in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, May 2019, pp. 291–300. [Online]. Available: <https://ieeexplore.ieee.org/document/8804457/>
- [14] K. H. Bennett and V. T. Rajlich, “Software maintenance and evolution: a roadmap,” in *Proceedings of the Conference on the Future of Software Engineering*, 2000, pp. 73–87.
- [15] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski, “Metrics and laws of software evolution—the nineties view,” in *Proceedings Fourth International Software Metrics Symposium*. IEEE, 1997, pp. 20–32.
- [16] S. M. Jain, “Hugging Face,” in *Introduction to Transformers for NLP*. Berkeley, CA: Apress, 2022, pp. 51–67. [Online]. Available: https://link.springer.com/10.1007/978-1-4842-8844-3_4
- [17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Huggingface’s transformers: State-of-the-art natural language processing,” *arXiv preprint arXiv:1910.03771*, 2019.
- [18] M. Mitchell, S. Wu, A. Zaldívar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” in *Proceedings of the conference on fairness, accountability, and transparency*, 2019, pp. 220–229.
- [19] K. Shivashankar and A. Martini, “Maintainability Challenges in ML: A Systematic Literature Review,” in *2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Gran Canaria, Spain: IEEE, Aug. 2022, pp. 60–67. [Online]. Available: <https://ieeexplore.ieee.org/document/10011474/>
- [20] J. Bogner, R. Verdecchia, and I. Gerostathopoulos, “Characterizing Technical Debt and Antipatterns in AI-Based Systems: A Systematic Mapping Study,” in *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE, May 2021, pp. 64–73, arXiv: 2103.09783. [Online]. Available: <https://ieeexplore.ieee.org/document/9463054/>
- [21] Y. Tang, R. Khatchadourian, M. Bagherzadeh, R. Singh, A. Stewart, and A. Raja, “An Empirical Study of Refactorings and Technical Debt in Machine Learning Systems,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, May 2021, pp. 238–250. [Online]. Available: <https://ieeexplore.ieee.org/document/9401990/>
- [22] M. Dilhara, A. Ketkar, and D. Dig, “Understanding Software-2.0: A Study of Machine Learning Library Usage and Evolution,” *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 4, pp. 1–42, Jul. 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3453478>
- [23] J. Leest, I. Gerostathopoulos, and C. Raibulet, “Evolvability of Machine Learning-based Systems: An Architectural Design Decision Framework,” in *2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C)*. L’Aquila, Italy: IEEE, Mar. 2023, pp. 106–110. [Online]. Available: <https://ieeexplore.ieee.org/document/10092638/>
- [24] A. Kathikar, A. Nair, B. Lazarine, A. Sachdeva, and S. Samtani, “Assessing the Vulnerabilities of the Open-Source Artificial Intelligence (AI) Landscape: A Large-Scale Analysis of the Hugging Face Platform,” in *IEEE Intelligence and Security Informatics*. Charlotte, NC, USA: IEEE, Oct. 2023.
- [25] A. Ait, J. L. C. Izquierdo, and J. Cabot, “HFCommunity: A Tool to Analyze the Hugging Face Hub Community,” in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Taipa, Macao: IEEE, Mar. 2023, pp. 728–732. [Online]. Available: <https://ieeexplore.ieee.org/document/10123660/>
- [26] V. R. B. G. Caldiera and H. D. Rombach, “The goal question metric approach,” *Encyclopedia of software engineering*, pp. 528–532, 1994.
- [27] A. Anonymous, “Replication Package for ‘What is the Evolution and Maintenance of Pre-Trained ML models on Hugging Face?’,” Nov. 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.10153155>
- [28] “HFapi Client,” https://huggingface.co/docs/huggingface_hub/package_reference/hf_api, Accessed: 01-02-2024.
- [29] E. B. Swanson, “The dimensions of maintenance,” in *Proceedings of the 2nd international conference on Software engineering*, 1976, pp. 492–497.
- [30] M. U. Sarwar, S. Zafar, M. W. Mkaouer, G. S. Walia, and M. Z. Malik, “Multi-label classification of commit messages using transfer learning,” in *2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2020, pp. 37–42.
- [31] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [32] D. M. Blei, A. Y. Ng, and M. I. Jordan, “Latent dirichlet allocation,” *Journal of machine Learning research*, vol. 3, no. Jan, pp. 993–1022, 2003.
- [33] M. Röder, A. Both, and A. Hinneburg, “Exploring the space of topic coherence measures,” in *Proceedings of the eighth ACM international conference on Web search and data mining*, 2015, pp. 399–408.
- [34] J. Coelho, M. T. Valente, L. Milen, and L. L. Silva, “Is this github project maintained? measuring the level of maintenance activity of open-source projects,” *Information and Software Technology*, vol. 122, p. 106274, 2020.
- [35] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, “DbSCAN revisited, revisited: why and how you should (still) use dbSCAN,” *ACM Transactions on Database Systems (TODS)*, vol. 42, no. 3, pp. 1–21, 2017.
- [36] P. E. McKnight and J. Najab, “Mann-whitney u test,” *The Corsini encyclopedia of psychology*, pp. 1–1, 2010.
- [37] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [38] E. L. Oreamuno, R. F. Khan, A. A. Bangash, C. Stinson, and B. Adams, “The state of documentation practices of third-party machine learning models and datasets,” *arXiv preprint arXiv:2312.15058*, 2023.
- [39] A. Bhat, A. Coursey, G. Hu, S. Li, N. Nahar, S. Zhou, C. Kästner, and J. L. Guo, “Aspirations and practice of ml model documentation: Moving the needle with nudging and traceability,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–17.
- [40] T. Zimmermann, S. Kim, A. Zeller, and E. J. Whitehead, “Mining version archives for co-changed lines,” in *Proceedings of the 2006 International Workshop on Mining Software Repositories*, ser. MSR ’06. New York, NY, USA: Association for Computing Machinery, 2006, p. 72–75. [Online]. Available: <https://doi-org.recursos.biblioteca.upc.edu/10.1145/1137983.1138001>
- [41] R. S. Pressman, *Software engineering: a practitioner’s approach*. Palgrave macmillan, 2005.
- [42] “Data Version Control · DVC,” <https://dvc.org/>, Accessed: 01-02-2024.
- [43] “DagsHub: The Home for Machine Learning Collaboration,” <https://dagshub.com/>, Accessed: 01-02-2024.
- [44] F. Lanubile, S. Martínez-Fernández, and L. Quaranta, “Training future ml engineers: a project-based course on ml ops,” *IEEE software*, 2023.
- [45] J. Tsay, A. Braz, M. Hirzel, A. Shinnar, and T. Mummert, “Aimmx: Artificial intelligence model metadata extractor,” in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 81–92. [Online]. Available: <https://doi.org/10.1145/3379597.3387448>